

• PROJET DE SOUTENANCE

Un seul système pour réserver, accueillir et tracer.

Flutter gère le terrain, Laravel garde la logique métier, et le dashboard rend les actions lisibles en temps réel.

Flutter

Laravel

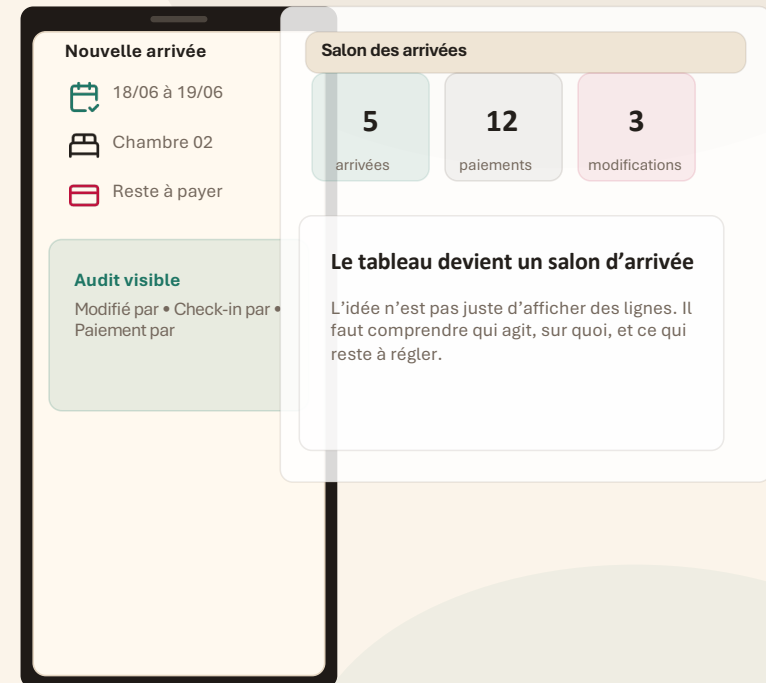
Dashboard

Ce que le projet évite

Les doublons de saisie, les réservations sans contexte, et les actions qu'on ne peut plus expliquer après coup.

Ce que l'interface montre

Qui a pris, qui a fait le check-in, qui a encaissé, qui a modifié, et combien il reste à payer.



• ARCHITECTURE

Trois blocs, mais un seul flux métier qui tient la route.

L'app terrain, le backend Laravel et le dashboard partagent la même réservation. Ça évite les vues parallèles qui se contredisent.



Flutter

Réserver, modifier, check-in, ajouter une chambre ou un supplément. Le terrain doit aller vite, sans écrire de code contourné.

Ce que le mobile manipule

Client, chambres, dates, suppléments, paiement simple.



Laravel

C'est là que la vérité métier vit: audits, règles d'édition, états de réservation, historiques clients et facturation.

API



Services principaux

BookingService, PMSController, audit des actions.



Dashboard

Le tableau de bord découpe le salon d'arrivée pour qu'on voie immédiatement l'état de chaque étape sans fouiller les réservations.

UI



Lecture métier

Pris par, check-in, paiement, acompte, reste à payer.

Ce que ça change

On ne duplique pas la logique dans trois endroits. Une action est créée une fois, puis elle remonte partout avec la même signification.

Ce que ça rend possible

Une réservation modifiée après arrivée reste lisible, mais les champs client ne bougent plus n'importe comment.

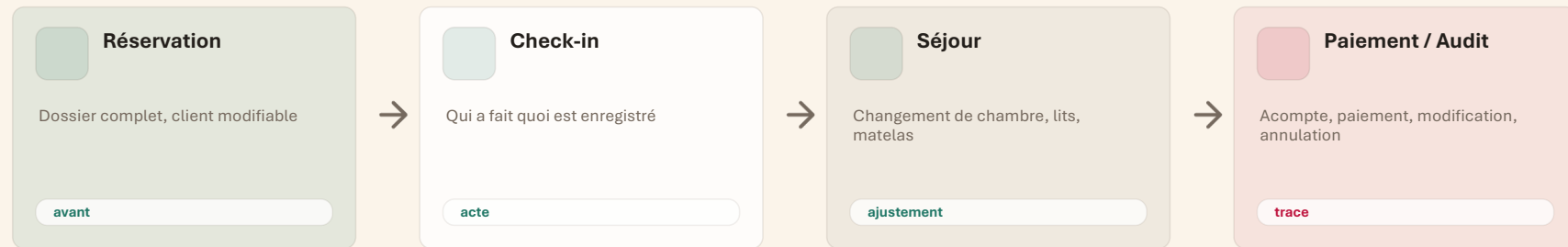
Ce que le dashboard exploite

Les mêmes données servent au salon des arrivées, à l'historique complet et au tri par séjour, solde ou montant.

• FLUX MÉTIER

Le point dur, c'est l'après check-in: encore bouger, sans casser le dossier client.

J'ai séparé ce qui doit rester figé de ce qui peut encore évoluer. C'est là que le projet devient vraiment hôtelier, pas juste "réservation + formulaire".



Avant

Toutes les informations restent modifiables. On peut corriger une chambre, des dates, des suppléments, sans bloquer inutilement.

Après check-in

Le client ne doit plus être édité. On garde les options opérationnelles, pas l'identité du dossier.

Ce qui est tracé

Check-in, paiement, acompte, modification et annulation. Chaque action prend un auteur et un contexte.

• TROIS BLOCAGES

Trois vrais problèmes de code ont forcé des corrections nettes.

Pas des “petits bugs”. Des endroits où la logique métier, l’historique ou la lisibilité pouvaient partir de travers.



1. Réservation bloquée trop tôt

Le code refusait encore de modifier la réservation alors qu’elle venait d’arriver. On perdait de la souplesse sur une opération courante.

Correction

J’ai séparé les champs client et les options. Après check-in, seule la partie opérationnelle reste ouverte.

```
if (status === 'arrive') {  
  allowRoomChanges = true;  
  allowGuestEdit = false;  
}
```



2. Aucune trace exploitable

Les actions existaient dans le flux, mais on ne savait pas clairement qui avait pris le paiement, validé le check-in ou modifié le dossier.

Correction

J’ai ajouté une table d’audit et branché les contrôleurs dessus pour enregistrer l’auteur, le rôle et les détails.

```
ReservationAudit::create([  
  'action' => 'payment',  
  'actor_name' => $user,  
]);
```



3. Dashboard trop vague

Un statut unique ne suffisait pas. Il fallait voir check-in, paiement, acompte, total et reste à payer sans deviner.

Correction

J’ai éclaté l’affichage en colonnes concrètes et ajouté le tri sur l’historique complet.

```
sortClientHistoryByKey('balance_amount_aria  
sortClientHistoryByKey('check_in_date')
```

Lecture rapide

Chaque correction répond à un vrai angle mort: règles d’édition, traçabilité, ou lisibilité métier.

• RÉSULTAT

Le système raconte enfin le séjour au lieu de juste stocker des lignes.

On a un parcours plus sûr pour l'équipe, une lecture plus claire pour le dashboard, et un historique qui sert vraiment quand il faut vérifier.



Plus sûr

L'édition post check-in reste possible, mais dans un cadre propre.



Plus lisible

Check-in, paiement, acompte et reste à payer apparaissent séparément.



Plus traçable

Chaque action importante laisse une empreinte avec auteur et contexte.

Ce qu'il faut retenir

Le projet n'a pas cherché à empiler des écrans. Il a remis de l'ordre dans la circulation de l'information: qui agit, quand, sur quoi, et ce qui reste à faire. C'est ça qui rend l'outil exploitable au quotidien.

Si vous voulez, je peux maintenant vous montrer le parcours en démo ou passer aux questions.

Merci.